

# AForce OS — Engineering Brief

## AForce OS — Engineering Brief for Provisional Patent Application

**Prepared for:** Outside counsel, AForce IP filing **Application:** AForce OS mobile application (Expo / React Native) **Date:** April 29, 2026 **Companion artifacts:** Patent figures FIG. 1 – FIG. 10 ( attached\_assets/patent/fig{1..10}-\*.svg )

This brief documents what AForce engineering has built and shipped in the AForce OS mobile application as of the date above. Every system below is implemented in production code, not vaporware. Each section cites exact file paths and line numbers in the repository so counsel can drop them directly into the specification or hand them to a technical reviewer.

The five systems counsel asked about — **Mobile Application**, **Scoring Engine**, **Deterministic AI Layer**, **Autopilot**, and **Autoscan** — are described in that order. Each maps to one of the attached patent figures (noted at the top of each section). Wireframes referenced throughout are also live, interactive iframes on the engineering canvas (AForce OS app preview, Recovery Card unlocked/locked, Hydration Score Band, Hero band).

### 1. Mobile Application

**Maps to:** FIG. 1 (System Architecture), FIG. 7 (Mobile App Architecture), FIG. 6 (Wireframes) **Repository root:** artifacts/aforce-os/ **Stack:** Expo (React Native), expo-router for file-based navigation, Clerk for auth, Postgres + Drizzle on the backend ( artifacts/api-server/ ), WebSocket for multi-device sync.

#### 1.1 Navigation skeleton

File-based routing through expo-router . Root Stack wraps a persistent bottom Tabs layout for the main experience.

| Route                             | File                                              | Purpose                                                                      |
|-----------------------------------|---------------------------------------------------|------------------------------------------------------------------------------|
| / (tab)                           | app/(tabs)/index.tsx                              | Hydration Control Center — animated status orb, AI command, prediction strip |
| /check (tab)                      | app/(tabs)/check.tsx                              | Performance Signals — biometric snapshot                                     |
| /protocol (tab)                   | app/(tabs)/protocol.tsx                           | Active recovery protocols                                                    |
| /store (tab)                      | app/(tabs)/store.tsx                              | Product shopping (RTD, sticks, canister)                                     |
| /profile (tab)                    | app/(tabs)/profile.tsx                            | User settings, HealthKit integration, Protocol Tools                         |
| /scan (modal)                     | app/scan.tsx                                      | Camera-based product scanning (Autoscan)                                     |
| /heat                             | app/heat.tsx                                      | Heat-risk prediction dashboard                                               |
| /sweat                            | app/sweat.tsx → screens/SweatCalculatorScreen.tsx | Sweat Intelligence v2 calculator (drives Autopilot)                          |
| /competition                      | app/competition.tsx                               | GPS-based territory map                                                      |
| /subscription                     | app/subscription.tsx                              | Premium plan management                                                      |
| /(auth)/sign-in , /(auth)/sign-up | app/(auth)/*                                      | Clerk-backed auth                                                            |

#### 1.2 State management

**Pattern:** Sliced React Context + useReducer . The monolithic AppState is projected through memoized slice contexts so unrelated UI sections do not re-render on unrelated state changes.

| Layer              | File                                    | Responsibility                                                                            |
|--------------------|-----------------------------------------|-------------------------------------------------------------------------------------------|
| Store orchestrator | store/useAppStore.tsx                   | Owns AppState , dispatch, periodic 30s server polling, WebSocket subscription             |
| Reducer (pure)     | store/appStoreReducer.ts                | All state transitions: intake logging, timer ticks, autopilot, engine refreshes           |
| Slice projections  | store/slices.tsx                        | EngineSlice , UserSlice , SocialSlice , CycleSlice , InventorySlice , SweatAutopilotSlice |
| Type contract      | store/appStoreTypes.ts , types/index.ts | AppState , UserState , ScoreEngineOutput , IntakeEvent , InventoryState , SweatAutopilot  |

### 1.3 Server / data layer

| Component         | File                                        | Notes                                                                                                                                                   |
|-------------------|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| API client        | lib/api.ts                                  | Typed fetch wrapper, persists x-device-id for anonymous-then-merged sessions                                                                            |
| Real API service  | services/realApi.ts                         | Normalizes Postgres rows into UserState and <b>re-runs the client-side scoring engine on every response</b> so UI always shows a freshly computed score |
| Realtime channel  | services/realApi.ts#subscribeToStateUpdates | WebSocket push from api-server for multi-device sync                                                                                                    |
| Local persistence | AsyncStorage                                | deviceId , Clerk tokenCache , aforce.subscription flag                                                                                                  |

### 1.4 Hardware integrations

| Surface         | File                                                              | Notes                                                                                                            |
|-----------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Voice           | services/voiceService.ts + voiceTemplateEngine.ts                 | Classifies transcripts into intents ( LOG_INTAKE , GET_STATUS , etc.) and renders brand-aligned spoken responses |
| Camera          | services/hydrationScanService.ts , components/CameraScanModal.tsx | expo-camera CameraView reading ean13 , upc_a , qr , code128                                                      |
| Apple HealthKit | services/appleHealth.ts                                           | restingHeartRate , hrvSdnn , sleepHoursLastNight fed into the scoring engine                                     |
| Auth            | components/ClerkAuthBridge.tsx                                    | Bridges Clerk session → internal store; (tabs) layout redirects unauthenticated users                            |
| Entitlement     | hooks/useEntitlement.ts                                           | Feature gating (subscriber vs. free)                                                                             |

### 1.5 Wireframes (live on canvas)

Counsel can view the actual rendered screens — not static mockups — in the engineering canvas:

| Canvas shape ID         | What it shows                   | Source                                                                     |
|-------------------------|---------------------------------|----------------------------------------------------------------------------|
| artifact: .../aforce-os | Full live AForce OS app         | artifacts/aforce-os/                                                       |
| recovery-unlocked       | Recovery Card — subscriber view | artifacts/mockup-sandbox/src/components/mockups/recovery-mode/Unlocked.tsx |
| recovery-locked         | Recovery Card — payoff          | .../recovery-mode/Locked.tsx                                               |
| score-band              | Hydration Score Band component  | .../score-band/Default.tsx                                                 |

| Canvas shape ID | What it shows                       | Source                  |
|-----------------|-------------------------------------|-------------------------|
| score-band-hero | Hydration Score Band — hero variant | .../score-band/Hero.tsx |

## 2. Scoring Engine

**Maps to:** FIG. 8 (Scoring Engine) **Primary file:** artifacts/aforce-os/utls/scoringEngine.ts **Supporting files:** services/hydrationScoreService.ts , types/index.ts , store/slices.tsx#useEngineSlice

The Scoring Engine is a **pure function** that takes a `UserState` snapshot and returns a `ScoreEngineOutput`. It runs both server-side (during state writes) and client-side (on every API response, for sub-network-latency UI updates). Because it is pure, the same input snapshot always produces the same output — this is the foundation of the determinism claim in §3.

### 2.1 Inputs ( `UserState` , `types/index.ts` )

- `intakeEvents[]` — recent logs with pre-computed absorption impact
- `ozConsumedToday` , `ozTarget` — base hydration progress
- `aforceUnitsToday` — count of AForce-specific intakes (drives the bonus term)
- `lastIntakeTime` — timestamp for decay and recovery-momentum calculations
- `heatLoad` , `sweatRate` , `activityLevel` — environmental + physiological stressors (0-10)
- `urineSignal` — biomarker proxy (1-8 scale)
- `symptomState` , `symptoms[]` — severity bucket + specific symptom IDs
- `appleHealth` — `hrvSdnn` , `sleepHoursLastNight` from HealthKit
- `clutchActive` — boolean; multiplies decay during high-intensity windows
- `socialMode` — alcohol mitigation state (drinks, ABV, food)

### 2.2 Output ( `ScoreEngineOutput` , `types/index.ts:256` )

```
{
  score: number,           // 0-100
  performanceState: { level, color, pulseSpeed },
  pulseConfig: { waveBehavior, animations },
  breakdown: ScoreContribution[], // {id, label, delta, maxMagnitude, hint}
  prediction: ScorePrediction,    // {decayPerMinute, minutesToDepleted, label}
  command: AICommand,            // {action, explanation, urgencyLevel, estimatedImpact}
  social: SocialRollup           // BAC, impairment level, safety prompts
}
```

### 2.3 Score formula ( `scoringEngine.ts:363` )

```
score = base_intake
+ aforce_bonus
+ recency
+ consistency
+ context
+ recovery_momentum
+ symptom_penalty      (negative)
+ urine_penalty        (negative)
+ output_stress        (negative)
+ sleep_carry
+ recovery_signal
+ confirmation
```

Each term is a bounded contribution with its own clamp and weight, exposed in the `breakdown` array so the UI can render an audit (the "Why this score?" sheet in `components/ScoreBreakdownSheet.tsx`).

### 2.4 Absorption model ( `hydrationScoreService.ts:25` )

- **Plain water:** 60 % immediate, 40 % over 12.5 min

- **AForce:** 70 % immediate, 30 % over 25 min (faster onset because of structured-water + mineral pairing)
- **Cap:** at most 1.5 units per rolling 20-min window; excess is 75 % efficient (models gut absorption ceiling)

## 2.5 Continuous decay ( `scoringEngine.ts:166-202` )

```
BaseDecay = 0.4 × (weight_lb / 150) + 0.1 × activity_level
+ heatModifier(temp_C > 25)
+ humidityModifier(RH > 50%)
× (clutchActive ? 1.3 : 1.0)
```

Decay is applied per-minute and surfaces in the UI as the prediction string ("Drops to DEPLETED in 1 h 12 m").

## 2.6 Score bands ( `scoringEngine.ts:38-42` )

| Band              | Range    | Default recheck interval ( <code>scoringEngine.ts:444-447</code> ) |
|-------------------|----------|--------------------------------------------------------------------|
| <b>PEAK</b>       | 90 - 100 | ≥ 20 min                                                           |
| <b>BALANCED</b>   | 75 - 89  | ≥ 15 min                                                           |
| <b>RECOVERING</b> | 60 - 74  | ≥ 10 min                                                           |
| <b>DEPLETED</b>   | 0 - 59   | ≥ 5 min                                                            |

These bands are overridden by the Autopilot system (§4) when an active sweat session is in progress.

## 2.7 UI consumption

Driven exclusively by `useEngineSlice()` ( `store/slices.tsx` ). Primary consumers:

- `components/StatusPulseOrb.tsx` — uses `pulseConfig` for the animated orb
- `components/AnimatedScore.tsx` — counts the score up/down
- `components/ScoreBreakdownSheet.tsx` — renders `breakdown[]`
- `components/home/OrbSection.tsx` — renders `command.action` + `command.explanation`
- Home screen "Prediction Strip" — renders `prediction.label`

# 3. Deterministic AI Layer

**Maps to:** FIG. 9 (Deterministic AI Layer)**Primary file:** `artifacts/aforce-os/utls/scoringEngine.ts` (command generator at lines 537-591) **Supporting:** `services/bacEstimationService.ts`, `services/legalSafetyService.ts`

## 3.1 What "Deterministic" means here

Conventional consumer health apps make recommendations through probabilistic LLM calls. The AForce Deterministic AI Layer is the opposite: an **ordered, pure-function rule pipeline** with **no remote inference and no nondeterministic LLM calls** in the safety- and performance-critical path. The same `UserState` snapshot always produces the same `AICommand`. This matters for three reasons that the application leans on as differentiators:

1. **Auditability.** Every output can be reproduced offline by replaying the snapshot through the same code. The `breakdown[]` array exposes every contributing term.
2. **Safety-class behavior.** Alcohol-mitigation recommendations ("DO NOT DRIVE", "PULL FROM ROTATION") cannot be subject to model drift or hallucination.
3. **Edge-first.** Decisions are computed locally on the device (or in the same reducer function on the server). The app degrades gracefully offline.

## 3.2 Rule pipeline (Stage 3)

The pipeline (FIG. 9) runs in this order, top-down, on each engine refresh:

1. **Mode classification** — Is the user in a *Social Mode* override (alcohol present)? If yes, skip the standard ladder; the Social Mode rule set takes precedence ( `scoringEngine.ts:458-531` ). Otherwise, use the standard PEAK / BALANCED / RECOVERING / DEPLETED ladder from §2.6.
2. **Penalty application** — Symptom severity, urine-color signal, output stress, and HRV/sleep deficits each apply a bounded negative delta.
3. **Bonus application** — AForce intakes, consistency streaks, recovery momentum, and confirmed intakes each apply a bounded positive delta.
4. **Decay projection** — Continuous decay model (§2.5) yields `prediction.minutesToDepleted` and the prediction label.
5. **Command generation** — `generateCommand()` (lines 537-591) selects a "WHAT + WHEN + OUTCOME" directive based on state, deficit, and inventory.
6. **Recheck cadence** — Default cadence by band (§2.6); if Autopilot is active, replaced by the autopilot interval (§4.3).

### 3.3 Social Mode override ( `scoringEngine.ts:458-531` )

When `socialMode.active === true` , the engine swaps in a parallel ruleset that:

- Estimates BAC via the **Widmark approximation** ( `services/bacEstimationService.ts:92-147` ), parameterized by sex, weight, drinks consumed, ABV, time elapsed, and food intake.
- Maps BAC to one of five **impairment levels** — LOW , ELEVATED , MODERATE , HIGH , CRITICAL ( `services/legalSafetyService.ts:28-37` ).
- Issues escalating commands and surfaces transportation / legal safety language. The disclaimer copy is locked and globally safe.

### 3.4 Guardian Risk Engine and Clutch Strategy

Two specialized branches sit on top of the core pipeline:

- **Guardian Risk Engine** ( `scoringEngine.ts:728-748` ) — applies an additional risk overlay when symptoms + heat + low HRV co-occur, escalating the command urgency and tightening the recheck cadence.
- **Clutch Strategy** ( `scoringEngine.ts:790-838` ) — when `clutchActive` is set (user has marked a high-intensity competitive window), the engine multiplies decay by 1.3, prefers AForce-led commands, and shortens cadence.

### 3.5 Outputs

- `command.action` — short imperative ("16 oz + 2 sticks now")
- `command.explanation` — rationale shown beneath the command
- `command.urgencyLevel` — low | medium | high | critical
- `command.estimatedImpact` — predicted score delta if executed
- `pulseConfig` — drives the orb animation so the user can read state at a glance
- `recheckMinutes` — fed to the timer (overridden by Autopilot when applicable)

---

## 4. Autopilot

**Maps to:** FIG. 10 (Autopilot / Autoscan) **Engine file:** `artifacts/aforce-os/services/sweatRateEngine.ts` **Wiring:** `screens/SweatCalculatorScreen.tsx` , `store/appStoreReducer.ts` , `hooks/useHeatGuard.ts` , `components/RiskTimerDisplay.tsx`

### 4.1 What Autopilot is

Autopilot is a **physiologically-paced recheck cadence** that takes over the timer for a fixed window (4 h) following a measured sweat session. Instead of using the band-default cadence from §2.6, the timer is driven by the user's just-measured **hydration deficit %** so high-deficit athletes get checked on more aggressively for the duration of the recovery window.

### 4.2 Trigger

The user runs the Sweat Calculator in any of three modes (Quick / Precision / Estimate) and taps Calculate. `commitSession()` ( `SweatCalculatorScreen.tsx:167-172` ) dispatches `SET_SWEAT_AUTOPILOT` with the session's autopilot derivation.

### 4.3 Cadence ladder ( `sweatRateEngine.ts:489-493` , verified)

```
export function deriveAutopilot(deficitPct: number): SweatAutopilot {  
  if (deficitPct >= 4) return { intervalMin: 8, urgency: 'critical', recoveryWindowHours: 4 };  
  if (deficitPct >= 2) return { intervalMin: 12, urgency: 'high', recoveryWindowHours: 4 };  
  return { intervalMin: 20, urgency: 'moderate', recoveryWindowHours: 4 };  
}
```

| Deficit | Interval | Urgency  | Window |
|---------|----------|----------|--------|
| ≥ 4 %   | 8 min    | critical | 4 h    |
| ≥ 2 %   | 12 min   | high     | 4 h    |
| < 2 %   | 20 min   | moderate | 4 h    |

### 4.4 Reducer wiring ( `store/appStoreReducer.ts` )

`SET_SWEAT_AUTOPILOT` does three things atomically:

1. Stores `sweatAutopilot` (interval + urgency) and `sweatAutopilotSetAt` (timestamp, used to enforce the 4 h window).
2. **Resets `state.timerSeconds = autopilot.intervalMin × 60`** so the visible countdown immediately reflects the new cadence.
3. Clears any `pendingConfirmation` so the new cadence is not blocked by a stale prompt.

### 4.5 Consumption ( `hooks/useHeatGuard.ts:106-115` )

```
if (autopilotSession is active within 4h)  
  recheckIntervalMin = autopilot.intervalMin  
  recheckUrgency     = autopilot.urgency  
  autopilotActive    = true  
else  
  recheckIntervalMin = heatBandDefault  
  recheckUrgency     = heatBandUrgency  
  autopilotActive    = false
```

`components/RiskTimerDisplay.tsx` renders the resulting `timerSeconds` and shows an "URGENT" badge when state is DEPLETED or remaining time is < 5 min.

### 4.6 Hand-off and graceful exit

After 4 h, `useHeatGuard` falls back to band-default cadence — there is no need to explicitly "exit" Autopilot. The 4 h horizon is the **Sawka 2007** post-exercise rehydration window and was chosen so the cadence aligns with the period during which sweat-driven sodium and fluid replacement is physiologically meaningful.

### 4.7 Sodium positioning carried through Autopilot

The Sweat Calculator that triggers Autopilot also feeds the Recovery Intelligence card with two derived values that anchor AForce's "25 mg sodium is intentional" positioning:

- `aforceSodiumTotalMg = aforceSticks × AFORCE_SODIUM_PER_UNIT_MG (=25)` — `sweatRateEngine.ts:128, 509`
- `sodiumGapMg = max(0, sodiumLossMg – aforceSodiumTotalMg)` — the *intentional* gap, framed as "covered by structured-water absorption + marine minerals"

These values are visible in the user-facing audit copy ("125 mg delivered by 5 units; gap 1 723 mg covered by structured-water absorption") and are the basis of the comparison-table claims (AForce 25 mg / Gatorade ~270 mg / LMNT 1 000 mg / Liquid IV ~500 mg).

## 5. Autoscan (HydroScan)

**Maps to:** FIG. 10 (Autopilot / Autoscan) **Primary screen:** artifacts/aforce-os/screens/HydrationScanScreen.tsx **Camera:** components/CameraScanModal.tsx **Service layer:** services/hydrationScanService.ts , services/productRecognitionService.ts , services/comparisonEngine.ts **Catalog:** data/productDatabase.ts

### 5.1 What Autoscan is

A "scan-to-decide" pipeline. The user points the phone at any beverage barcode (or types one in). The system identifies the product, scores it against the user's *current physiological state* using the same engine described in §2/§3, and — if an AForce SKU would outperform the scanned product — issues a one-tap replacement recommendation.

### 5.2 Input modes

| Mode                | Used on                    | Mechanism                                                                                                                                                 |
|---------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Native barcode / QR | iOS, Android               | expo-camera CameraView ( CameraScanModal.tsx:38–48 ) reading ean13 , upc_a , qr , code128 . AForce-specific QR codes use the scheme aforce://product/<id> |
| Manual search       | All platforms              | Free-text product / brand / digit lookup                                                                                                                  |
| Mock scan tray      | Web preview, demo, testing | HydrationScanScreen.tsx:217–250 exposes a fixture tray of barcodes for one-tap simulation                                                                 |

### 5.3 Resolution pipeline ( services/productRecognitionService.ts )

1. Look the barcode up in the local BARCODE\_INDEX ( data/productDatabase.ts ) — instant, offline, zero network.
2. If miss, fall back to the **Open Food Facts API** ( productRecognitionService.ts:126 ) so any US beverage can be resolved even when not in the local catalog.
3. Normalize the resolved product into the local ScannedProduct schema (name, brand, category, hydration markers, electrolyte density, sugar level, mineral content).

### 5.4 Scoring ( services/hydrationScanService.ts:55–63 )

The product is scored on:

- **Hydration efficiency** (0–100 %) — mineral content + water content – sugar penalty
- **Fit score** (0–100) and **verdict** ( Optimal / Strong / Acceptable / Suboptimal / Avoid ) based on the user's current protocol context (Heat Stress, Recovery, Social, etc.) — i.e. the same UserState that drives §2/§3.

### 5.5 AForce comparison ( hydrationScanService.ts:70 , 93–101 )

The catalog is filtered to isAForce: true SKUs. The engine picks the best AForce product for the user's current state, computes its fit score, and compares to the scanned product. If aforceFitScore ≥ scannedFitScore + 4 , an **AForce Replacement Card** ( components/AForceReplacementCard.tsx ) is rendered on the result, including the situational headline, detail, and a specific command ("Take 1 AForce Stick now with 16 oz water").

### 5.6 Output ( ScanOutcome )

- product — resolved details
- efficiency — 0–100 % hydration efficiency
- fitScore + verdict — for the scanned product

- `aforceUplift` — if applicable: `{ aforceProduct, fitDelta, command }`
- `recommendation` — situational headline + detail + command, rendered by `components/ScanResultCard.tsx`

## 5.7 Why Autoscan is novel

Existing barcode-scanning beverage apps return *static* nutrition facts. Autoscan scores a product against a *live* user state derived from the same Deterministic AI Layer that drives the rest of the app, and replaces a static "facts" output with a contextual decision (drink it / don't / use AForce instead). The replacement decision is auditable because both fit scores are computed by the same pure scoring code.

## 6. Patent figure ↔ implementation cross-reference

| Figure  | Title                   | Maps to section  | Primary implementation file                                                                      |
|---------|-------------------------|------------------|--------------------------------------------------------------------------------------------------|
| FIG. 1  | System Architecture     | §1               | <code>artifacts/aforce-os/</code> , <code>artifacts/api-server/</code>                           |
| FIG. 2  | Hero Intake Flow        | §1 (home)        | <code>app/(tabs)/index.tsx</code> , <code>components/StatusPulseOrb.tsx</code>                   |
| FIG. 3  | Paywall Flow            | §1 (entitlement) | <code>hooks/useEntitlement.ts</code> , <code>app/subscription.tsx</code>                         |
| FIG. 4  | Social Mode Flow        | §3.3             | <code>scoringEngine.ts:458-531</code> , <code>services/bacEstimationService.ts</code>            |
| FIG. 5  | Voice Command Flow      | §1.4 (voice)     | <code>services/voiceService.ts</code> , <code>voiceTemplateEngine.ts</code>                      |
| FIG. 6  | Wireframes              | §1.5             | live iframes on canvas + <code>screens/*</code>                                                  |
| FIG. 7  | Mobile App Architecture | §1               | <code>app/_layout.tsx</code> , <code>store/useAppStore.tsx</code>                                |
| FIG. 8  | Scoring Engine          | §2               | <code>utils/scoringEngine.ts</code> , <code>services/hydrationScoreService.ts</code>             |
| FIG. 9  | Deterministic AI Layer  | §3               | <code>utils/scoringEngine.ts:537-591</code> , <code>:728-748</code> , <code>:790-838</code>      |
| FIG. 10 | Autopilot / Autoscan    | §4, §5           | <code>services/sweatRateEngine.ts:489-493</code> , <code>services/hydrationScanService.ts</code> |

## 7. Suggested claim hooks (engineering input only — not legal advice)

Counsel may want to consider these implementation details as candidate claim elements:

1. **Pure-function pipeline computing a per-minute decaying performance score** from a defined set of biometric, environmental, and behavioral inputs, with a server- and client-side identity invariant (same input ⇒ same output).
2. **Sweat-deficit-driven cadence override** that replaces a band-based recheck timer with a physiologically-derived interval ladder (8/12/20 min) for a bounded recovery window (4 h) following a measured sweat session.
3. **Beverage-recognition pipeline that scores any scanned product against a live user state** (not a static nutrition database) and conditionally surfaces a brand-specific replacement when the brand's fit score exceeds the scanned product's fit score by a configurable margin.
4. **Audit-on-output:** every score returned to the UI carries a `breakdown[]` array of bounded contributions so the user can inspect why the score is what it is.
5. **Mode-priority pipeline** in which a Social Mode ruleset (BAC + impairment) preempts the standard performance ladder and emits safety-class commands that the rest of the engine cannot override.



---

## 8. Verification

- TypeScript: `pnpm --filter @workspace/aforce-os typecheck` is clean as of this brief.
- Visual: AForce OS app preview, Recovery Card (Unlocked/Locked), and Hydration Score Band variants are live iframes on the engineering canvas; counsel can interact with them directly.
- Constants in this brief (25 mg sodium per AForce unit; 8 / 12 / 20 min autopilot ladder; 90 / 75 / 60 band thresholds) were re-verified by ripgrep against the source files cited above on the date of this brief.

---

*End of brief. For follow-up — additional cross-references, formula derivations, or extracts of specific source files — contact AForce engineering.*